

Architectures et Programmation Parallèles

Cuda

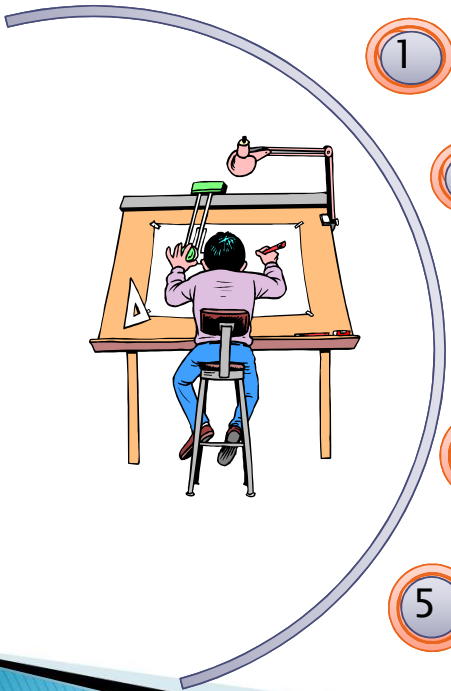
Section :

FIA2-GL

Prof. : Tahar ALIMI

Année Académique : 2024/2025

Plan



1

Introduction et domaines d'application

2

Architectures d'un GPU

3

Architecture logicielle CUDA

4

Programmation massive en CUDA

5

Coopération des threads et temps d'exécution en CUDA

Introduction (1/2)

➤ *Le parallélisme et les GPUs !*

- ❑ Le monde informatique a quasiment basculé vers le traitement parallèle depuis l'introduction des premiers processeurs multicœurs aux alentours de l'année 2005
- ❑ Parallèlement au développement vécu par la technologie des *CPUs* (Central Processing Units), les ordinateurs personnels ont commencé à se doter d'accélérateurs graphiques 2D afin d'améliorer l'affichage et la fluidité des systèmes d'exploitation graphiques au début des années 1990
- ❑ Au milieu des années 1990 (avec l'avènement de *SE* graphiques comme Windows 95 et la baisse des prix des PC), la demande de graphisme en 3D s'est accrue dans le monde de l'informatique personnelle (jeux vidéos)
- ❑ L'apparition de *GPU* (Graphics Processing Unit) permettant de programmer les traitements déclencha de nombreuses recherches sur la possibilité d'utiliser les circuits graphiques pour autre chose qu'un affichage graphique OpenGL ou DirectX.

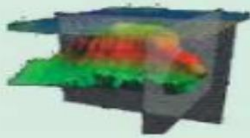
Introduction (2/2)

- ❑ L'idée c'est de pouvoir utiliser le GPU pour exécuter des traitements généraux, où ce qui est connu sous le nom de *GPGPU* (General-Purpose Computing on Graphics Processing Units)
- ❑ En novembre 2006, NVIDIA révéla le premier GPU (GeForce 8800 GTX) reposant sur l'architecture *CUDA* (Compute Unified Device Architecture), et dédié aux traitements généraux. Les langages associés sont *CUDA C/C++* et *CUDA Fortran*.



NVIDIA Tesla C2070 (pour Calcul Scientifique)

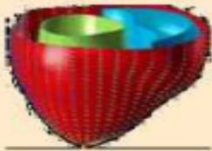
Traitements généraux avec CUDA (1/1)



**Computational
Geoscience**



**Computational
Chemistry**



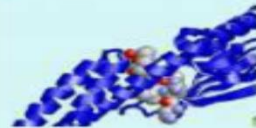
**Computational
Medicine**



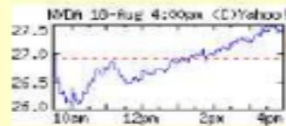
**Computational
Modeling**



**Computational
Physics**



**Computational
Biology**



**Computational
Finance**

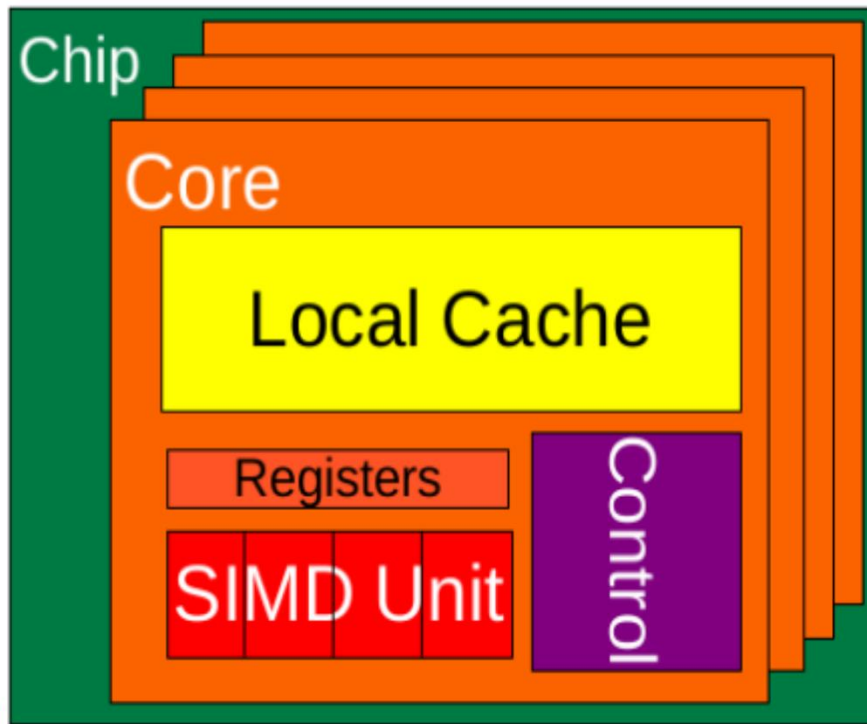


**Image
Processing**

GPU vs CPU(1/4)

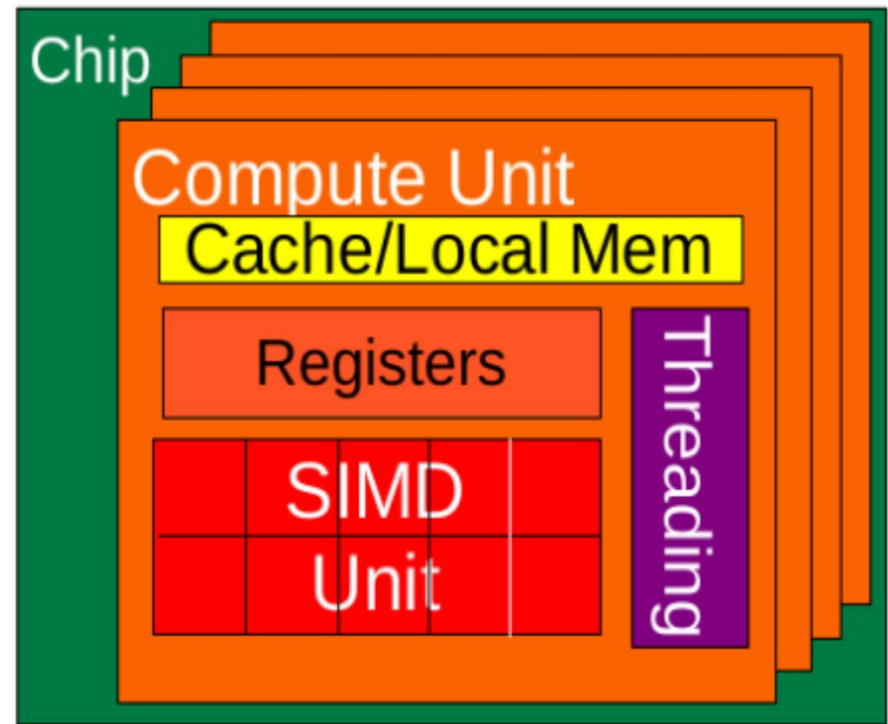
CPU

Latency Oriented Cores



GPU

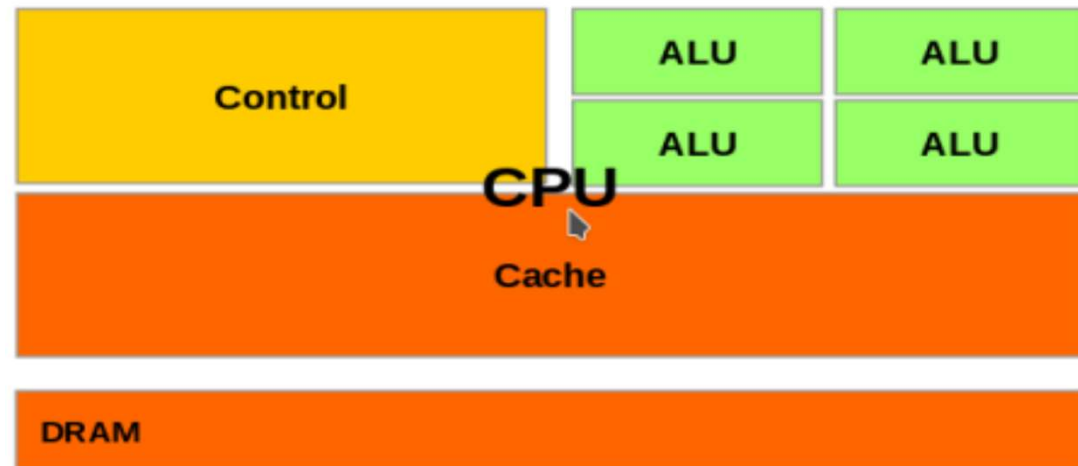
Throughput Oriented Cores



GPU vs CPU(2/4)

□ CPU : Conception Orientée Latence

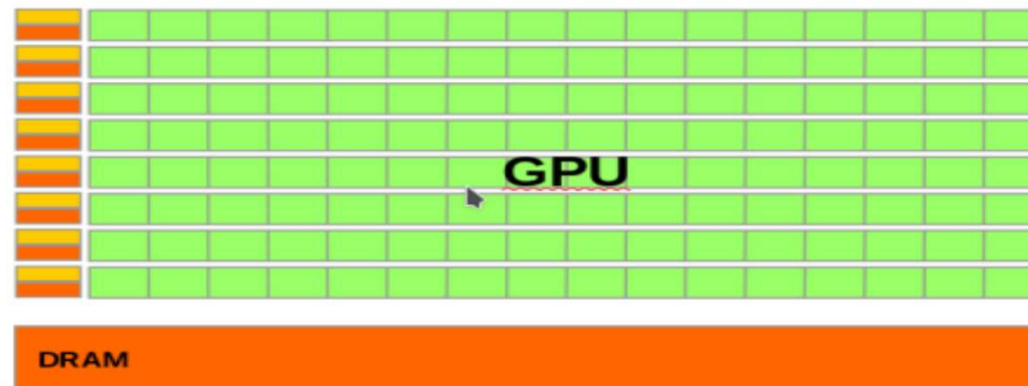
- Haute fréquence d'horloge
- Caches de grandes tailles
 - ⇒ Converti les accès à haute latence dans la mémoire en accès à basse latence dans le cache
- Système de contrôle sophistiqué
 - ⇒ Chargement de données pour réduire la latence dû à l'accès aux données
- Puissante unité arithmétique et logique (**ALU**: Arithmetic and Logic Unit)
 - ⇒ Réduction de la latence des opérations



GPU vs CPU(3/4)

❑ GPU : Conception Orientée Débit

- Fréquence d'horloge modérée
- Caches de petites tailles
 - ⇒ Pour maximiser le débit mémoire
- Contrôle simple
 - ⇒ Pas de chargement de données
- Unité arithmétique et logique (**ALU**) à faible consommation
 - ⇒ Nombreuses et à haute latence
- Nécessite un très grand nombre de threads pour que la latence soit tolérable



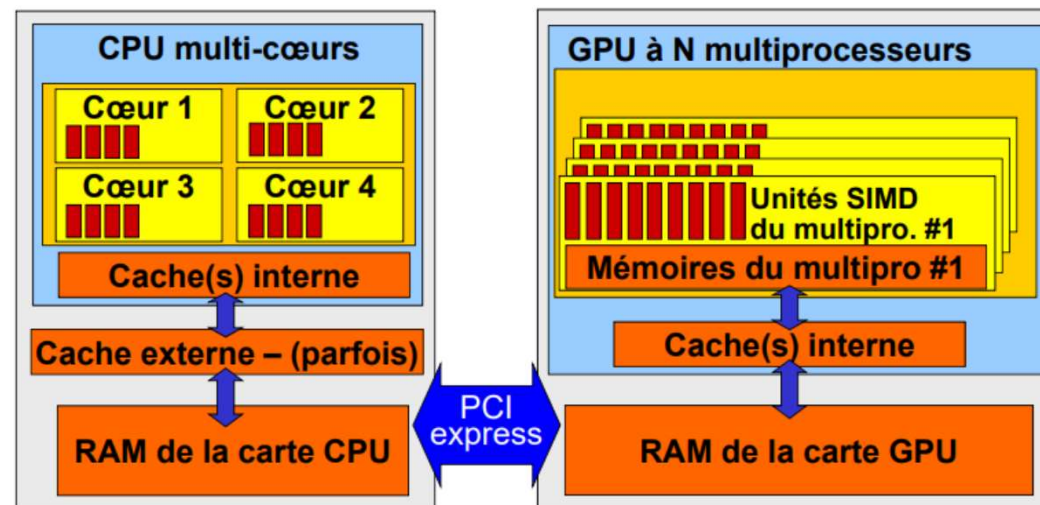
GPU vs CPU(4/4)

□ Bonne stratégie

- La stratégie gagnante est d'utiliser les deux

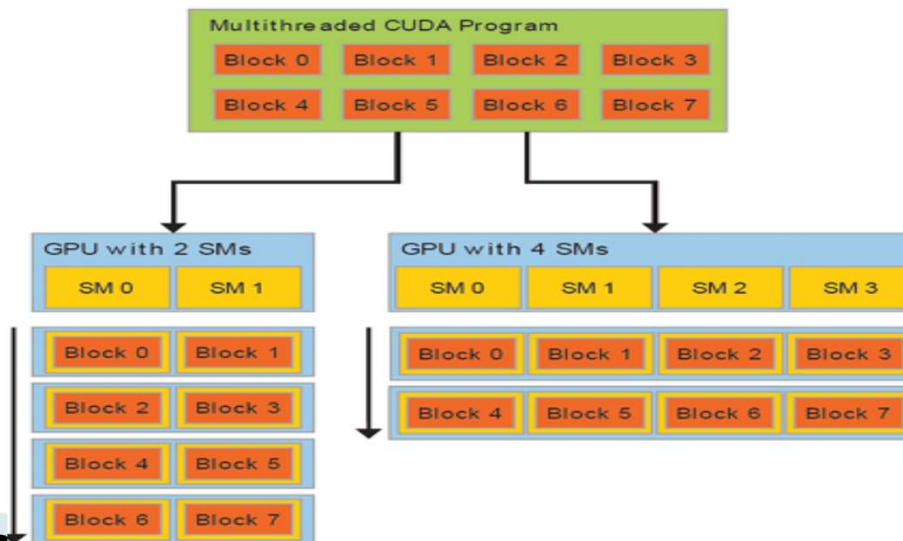
CPU	GPU
<i>CPUs</i> pour les parties séquentielles où la latence est critique	<i>GPUs</i> pour les parties parallèles où le débit est critique
<i>CPUs</i> peuvent être +10 fois plus rapides que les <i>GPUs</i> pour un code séquentiel	<i>GPUs</i> peuvent être +10 fois plus rapides que les <i>CPUs</i> pour un code parallèle

- Le *CPU* utilise le *GPU* comme un coprocesseur scientifique pour certains calculs adaptés aux architectures *SIMD*
- Le *CPU* et le *GPU* sont tous les deux des multicœurs et sont associés à des hiérarchies de mémoires



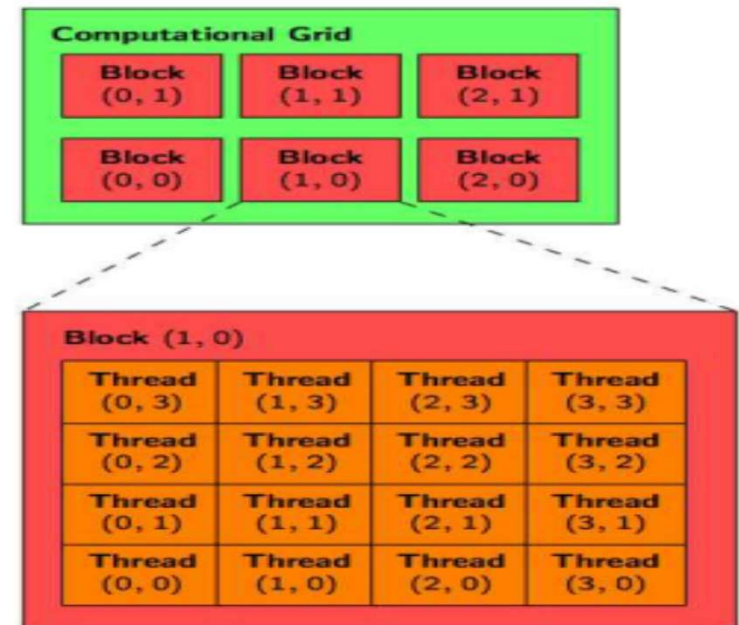
Architecture d'un GPU (1/1)

- Un *GPU* est un ensemble de N petites machines *SIMD* indépendantes et partageant une mémoire globale : N « multiprocesseurs »
- Chaque Multiprocesseur est un processeur autonome *SIMD* multi-thread
- Un *SIMD* correspond à un Streaming Multiprocessors (*SM*)
- Une hiérarchie de mémoires de tailles et de temps d'accès très variables
- Un ordonnanceur est une unité du *GPU* chargée de répartir les blocs (groupes de threads) à exécuter sur les multiprocesseurs
- Les threads d'un programme sont répartis sur des grilles contenant des *blocs* qui eux même contiennent des threads (les dimensions ne sont pas fixées)



CUDA (1/4)

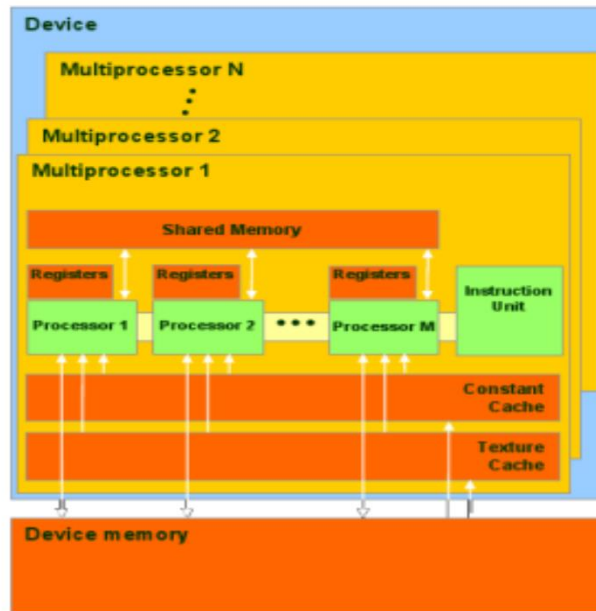
- **CUDA** : Compute Unified Device Architecture
- Programmation massivement parallèle en C sur cartes NVIDIA (Physique ou virtuelle sur le Cloud)
- Une **grid** est un tableau de “**thread blocks**”
- Chaque **thread block** est un tableau “**threads**”, chacun exécutant un clone (instance) du kernel
- Chaque **block** a un unique **blockId**
- Chaque thread a un unique **threadId** (dans un block donné)
- **CUDA** = Programme séquentiel en C avec des **kernels** parallèles :
 - ✓ Le code séquentiel s'exécute sur le host (**CPU**)
 - ✓ Le code parallèle s'exécute dans de nombreux device (**GPU**) threads sur plusieurs éléments de traitement



CUDA (2/4)

□ Les Mémoires

- Mémoire hôte : Située dans la mémoire vive du *CPU*. Inaccessible par les processus *CUDA*.
- Mémoire globale : Située dans la mémoire vive de la carte vidéo, accessible par les processus *CUDA* de tous les blocs.
- Mémoire partagée : Accessible par tous les threads d'un bloc, elle est très rapide
- Mémoire locale : Mémoire privée d'un thread



► espaces mémoire *off-chip* :

- **mémoire globale** : RW, jusqu'à qq GBytes, lente (qq 100 cycles d'horloge)

► espaces mémoire *on-chip* :

- **texture** : RO, physiquement en mémoire globale mais cachée
- **constant** : RO, 64kB/puce, très rapide (1-4 cycles)
- **partagée** : RW, très rapide (si conflits de banques évités), 16kB/multiprocesseur
- **registre** : RW, très rapide, 8-16kB/multiprocesseur

CUDA (3/4)

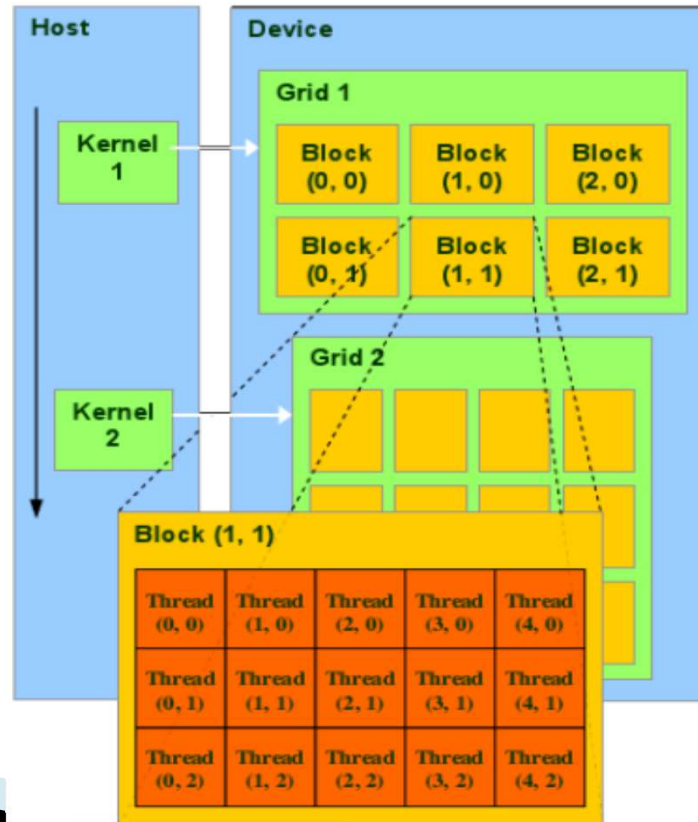
❑ Hiérarchie de mémoires et de threads

- Le *thread* est au niveau le plus bas de la hiérarchie
- Le *bloc* composé de plusieurs threads exécutées de manière concurrente
- La *grille* composée de plusieurs blocs de threads exécutés de manière concurrente
- La *mémoire locale* dédiée à chaque thread, per-thread local memory, est visible uniquement depuis le thread (cela concerne également des registres) ; Les registres sont sur le processeur et disposent de temps d'accès très rapide.
- La mémoire locale, dispose d'un *temps d'accès* plus lent que celui des registres.
- La *mémoire partagée* associée à chaque *bloc*, per-block shared memory, visible uniquement par tous les threads du bloc; Le bloc dispose de sa propre mémoire partagée et privée pour permettre des communications inter-threads rapides et de taille réglable
- La *mémoire globale*, «per-device global memory», utilisable par le «*device*».
- Une grille, «*grid*», utilise la mémoire globale.
- Cette mémoire globale permet de communiquer avec la *mémoire de l'hôte* et sert de lien de communication entre l'hôte et le *GPGPU* (General-Purpose Graphics Processor Unit)

CUDA (4/4)

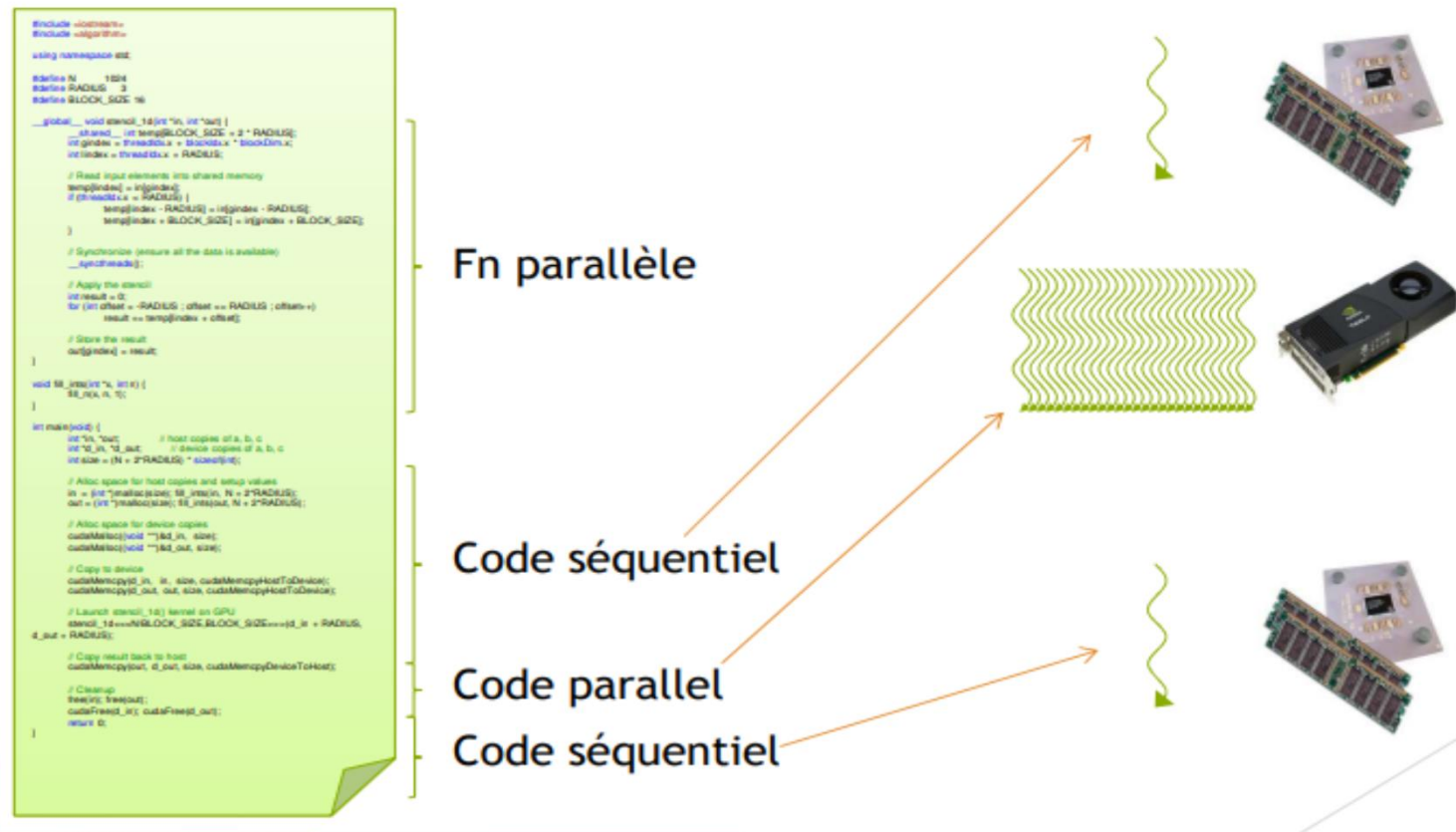
❑ Modèle de programmation CUDA

- Écrire un programme pour un pixel *thread*
- Instancier ce programme sur beaucoup de threads en parallèle
- Grille (logique) indexée de blocs de threads : *threadIdx* et *blockIdx* (variables intrinsèques)
- Les threads d'un même bloc ont accès à un espace mémoire partagée
- Les blocs sont indépendants, pas de synchronisation avec exécution dans un ordre indéfini



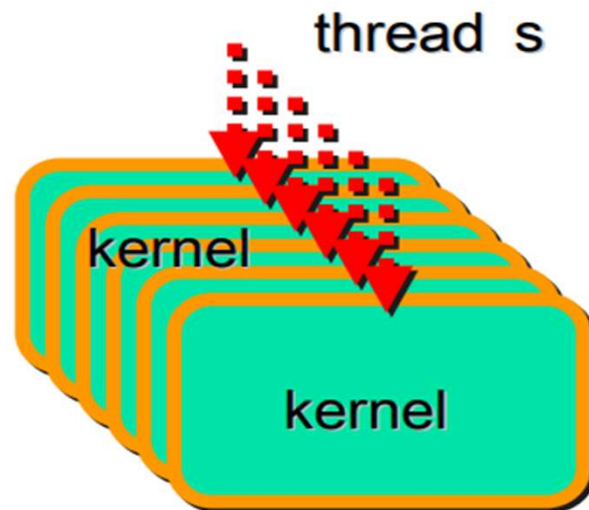
Hôte - Device (1/1)

- L'**hôte** est le **CPU** et sa mémoire, c'est lui qui demande au périphérique (le GPU) d'effectuer les calculs.
- **Device** : C'est le **GPU** et sa mémoire (device memory)
- Le programme est constitué d'une succession de codes séquentiels s'exécutant sur le Host et de codes parallèles s'exécutant sur le Device



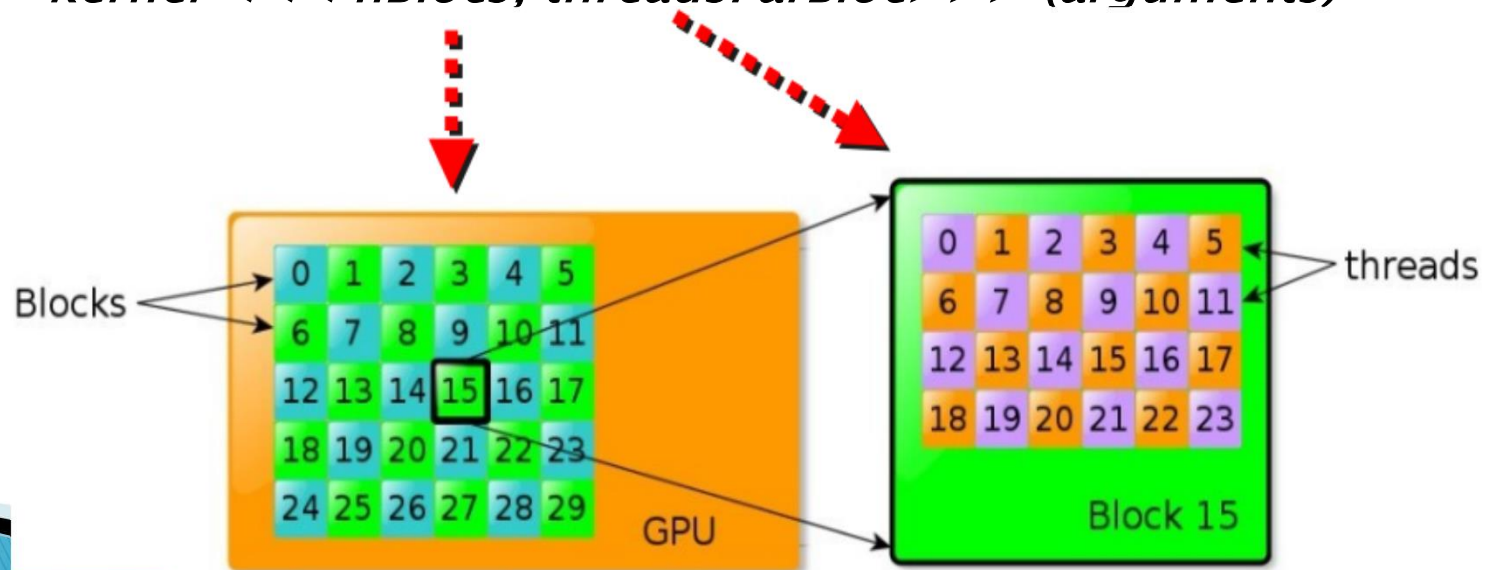
Kernel (1/3)

- Un *kernel* est une portion parallèle de code à exécuter sur le *Device*
- Chacune des instances du kernel s'appelle un thread
- Kernel = *Grille de blocs de threads* (Kernel = grid of thread blocks)
- La compilation se fait par le compilateur nvcc (NVidia Cuda Compiler)
- *Kernel* est exécutée sur le *GPU* et exige le type “*void*”



Kernel (2/3)

- Il en existe différents types, qualifiés de :
 - ✓ `__global__` : correspond à un kernel exécuté sur le GPU mais appelé par le CPU
 - ✓ `__device__` : correspond à un kernel exécuté et appelé par le GPU
 - ✓ `__host__` : correspond à une fonction exécutée et appelée par le CPU, c'est le mode de fonctionnement par défaut
- Un appel de kernel se fait en spécifiant 2 paramètres entre triples chevrons précédant les paramètres passés au kernel :
kernel <<< nBlocs, threadsParBloc >>> (arguments)

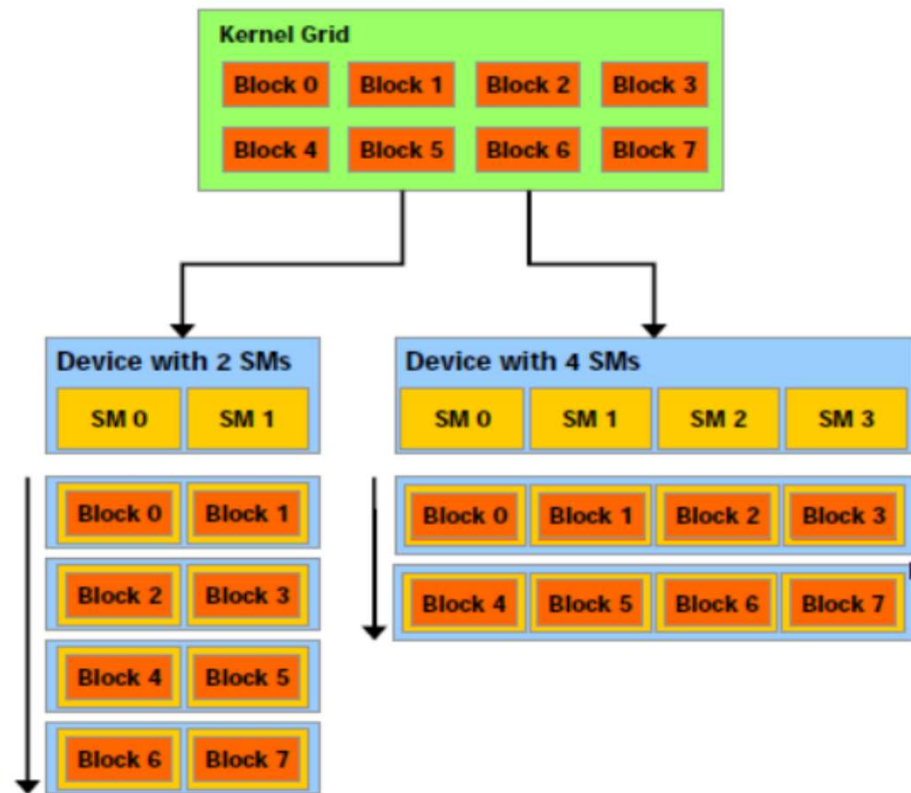


Kernel (3/3)

- *kernel* <<< *nBlocs*, *threadsParBloc*>>> (*arguments*);
- *nBlocs* est le nombre de subdivisions appliquées à la grille à calculer et est de type: dim3 – le cast à partir d'un entier N initialise le dim3 à {N, 1, 1}
- *threadsParBloc* indique le nombre de threads à exécuter simultanément pour chaque bloc, cette valeur est de type dim3
- Chaque kernel dispose de variables implicites en lecture seule (toutes de type dim3) :
 - ✓ *blockIdx* : index du bloc dans la grille,
 - ✓ *threadIdx* : index du thread dans le bloc,
 - ✓ *blockDim* : nombre de threads par bloc (valeur de *threadsParBloc*)

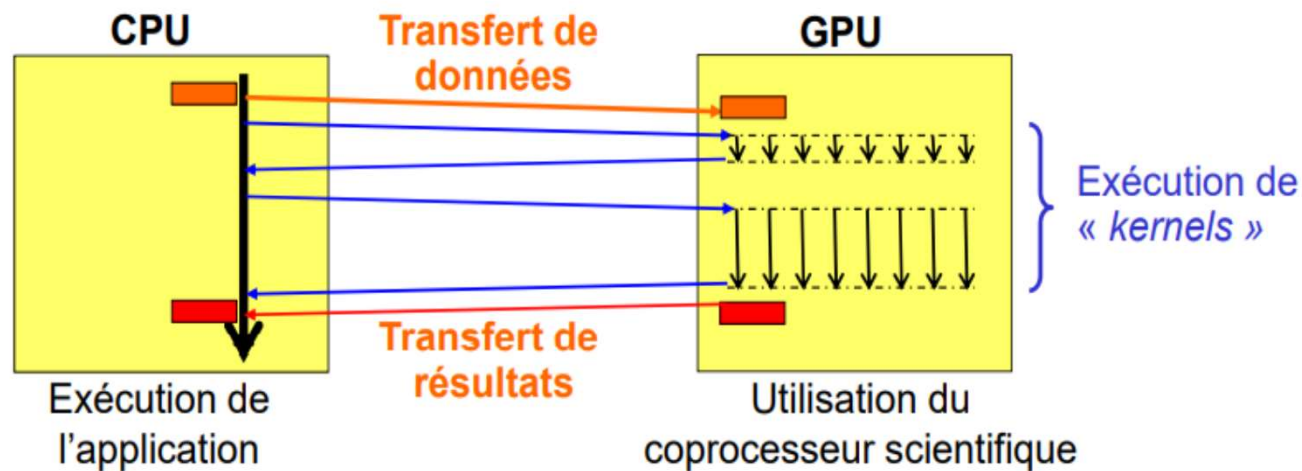
Blocks (1/1)

- Une grille est constituée de blocs
- Chaque bloc est constitué de threads
- Un bloc est un élément des calculs, dissociable d'autres blocs.
- Les blocs peuvent être exécutés parallèlement ou consécutivement
- Les threads ne peuvent communiquer qu'avec des threads du même bloc



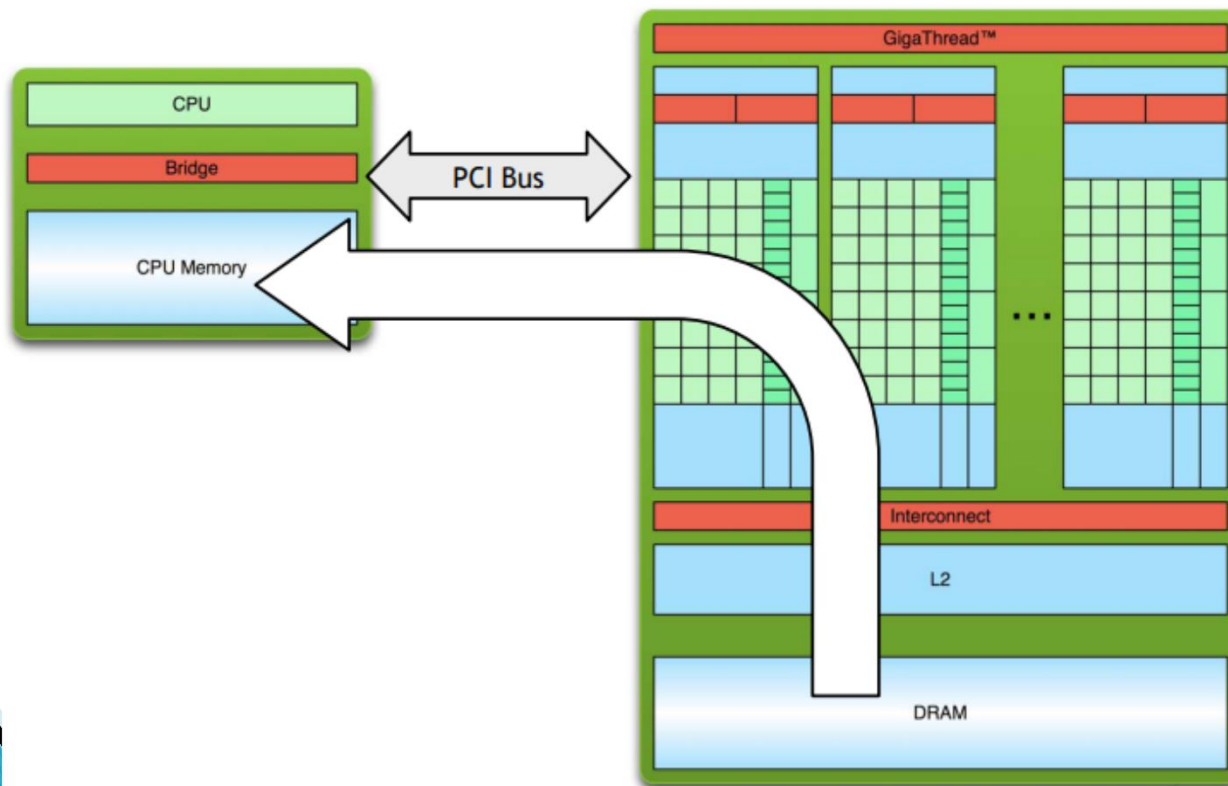
Principe d'exécution (1/1)

- On lance un programme *CPU* d'apparence classique : exécution de « kernels »
 - Il faut minimiser les transferts de données (pour être efficace).
 - On peut exécuter les « kernels » en mode bloquant (*synchrone*) ou non-bloquant (*asynchrone*) vis-à-vis du programme *CPU*
- ⇒ possibilité d'utiliser simultanément le *CPU* et le *GPU*



Calcul hétérogène (1/1)

- C'est le fait d'interroger le *CPU* et le *GPU*
 - 1) Copier les données d'entrée de la mémoire du *CPU* vers la mémoire du *GPU*
 - 2) Charger le Programme et l'exécuter sur le *GPU*
 - 3) Copier les résultats à partir de la mémoire du *GPU* vers la mémoire du *CPU*



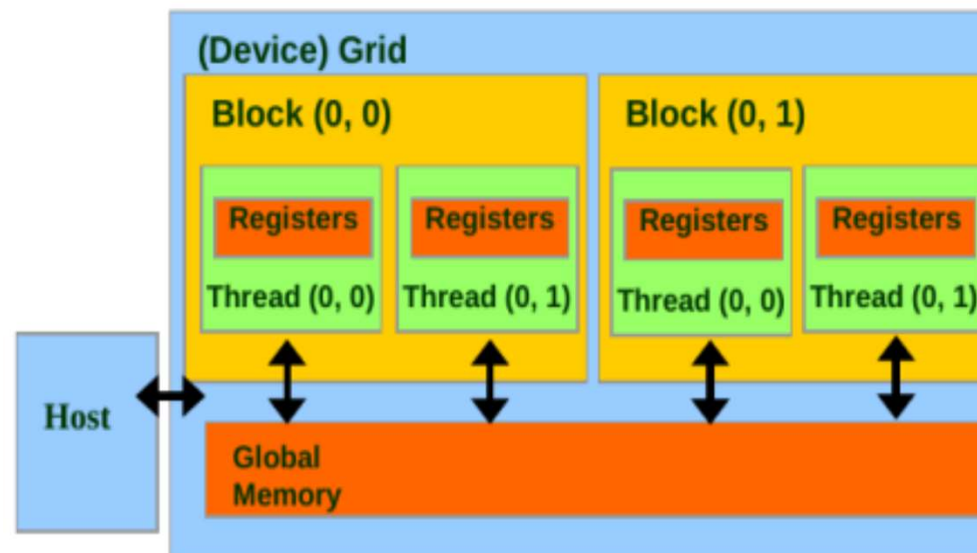
Allocation/Libération mémoire globale (1/1)

➤ **cudaMalloc :**

- ✓ Allouer un objet dans la mémoire globale du *GPU*
- ✓ Deux paramètres : Adresse du pointeur où sera allouée l'objet et la Taille (en bytes) de l'objet à allouer

➤ **cudaFree :**

- ✓ Libère l'objet dans la mémoire globale
- ✓ Un seul paramètre : Le pointeur vers l'objet à libérer



Transfert mémoire CPU-GPU (1/1)

➤ `cudaMemcpy()`

- ✓ Transfert de données vers/depuis la mémoire *GPU*
- ✓ Nécessite 4 paramètres : Pointeur de la destination, Pointeur de la source, Nombre de bytes à copier et le Type/Direction du transfert
- ⇒ *cudaMemcpyHostToDevice* / Copier du *CPU* vers *GPU*
- ⇒ *cudaMemcpyDeviceToHost* / Copier du *GPU* vers *CPU*
- ✓ ATTENTION : Le transfert est *asynchrone*

Premier programme CUDA (1/2)

➤ Le fameux programme "Hello.cu"

```
#include <stdio,h>
#include <cuda,h>
__global__ void hello_GPU()
{ // Le type void du kernel (fonction hello_GPU) est exigé
  printf("Hello world!\n");
}

int main()
{
  hello_GPU<<<1,1>>>>();
}
```

- L'appel de la fonction hello_GPU (kernel launch) s'est fait avec 1 seul bloc et 1 seul thread par bloc
- Mais ce programme ne fait rien : L'application *CPU* s'est terminée avec l'exécution du code *GPU*
- L'exécution s'est fait en mode asynchrone
- Le Host doit attendre le *Device* terminer sa tâche
- On doit synchroniser l'exécution : *cudaDeviceSynchronize()*

➤ Le mot clé CUDA C/C++ *__global__* indique une fonction qui :

- ✓ est appelée à partir du code host (*CPU*)
- ✓ et qui s'exécute sur le device (*GPU*)
- ✓ Cette fonction est généralement nommée Noyau (*Kernel*)

➤ *nvcc* sépare le code source en composantes hôte (host) et accélérateur (*device*)

- ✓ Les fonctions *Device* sont traitées par le compilateur NVIDIA (*nvcc*)
- ✓ Les fonction Host sont traitées par un compilateur standard du Host (e,g, gcc)

➤ Le *compilateur* et l'environnement d'exécution *CUDA* s'occupent de tous les détails de l'appel du code accélérateur à partir de l'*hôte*

Premier programme CUDA (2/2)

➤ Le fameux programme "Hello.cu"

```
#include <stdio,h>
#include <cuda,h>
__global__ void hello_GPU()
{
    printf("Hello world!\n");
}

int main()
{
    hello_GPU<<<1,1>>>();
    cudaDeviceSynchronize();
}
```

➤ Ce programme affiche une seule fois le message "Hello world!" : l'appel d'est fait avec 1 seul bloc et 1 thread par bloc

➤ hello_GPU<<<3,1>>>();

⇒ Affiche le message 3 fois : 3 blocs à raison d'un thread par bloc ($3 * 1 = 3$)

➤ hello_GPU<<<1,3>>>();

⇒ Affiche le message 3 fois : 1 seul bloc à raison de 3 threads par bloc ($1 * 3 = 3$)

Programmation massive avec CUDA (1/5)

- **CUDA** est conçu pour faire de la programmation massivement parallèle et non pas pour faire des appel à des noyaux qui ne font rien ou qui exécutent de simples traitements (Affichage simple, ...)
- Nous allons monter cela à travers un exemple de calcul de la somme de deux vecteurs d'entiers
- Généralement, il s'agit du parallélisme de données. C'est le même traitement qui est appliqué à des données différentes
- Un appel avec la syntaxe `<<< N, 1 >>>` permet de lancer N instances de la fonction noyau en parallèle
- L'ensemble des blocs est appelé grille
- Chaque instance de la fonction noyau peut référencer l'indice de son bloc en utilisant la syntaxe ***blockIdx.x*** (.x signifie qu'on utilise la représentation des blocs sur une dimension)
- Les différentes valeurs de ***blockIdx.x*** peuvent être utilisées pour accéder aux différents éléments d'un tableau
- Chaque ***bloc*** (ou instance de la fonction noyau) va s'occuper de l'addition des deux valeurs des vecteurs ayant pour indices le numéro du bloc en cours
- Les différentes additions se font en parallèle sur les unités de calculs de l'accélérateur
- Nous allons lancer autant de blocs (instances de la fonction noyau) qu'il y a d'éléments dans un vecteur

Bloc 0

```
c[0] = a[0] + b[0];
```

Bloc 1

```
c[1] = a[1] + b[1];
```

Bloc 2

```
c[2] = a[2] + b[2];
```

Programmation massive avec CUDA (2/5)

// Prog. En C

```
▶ #include<stdio.h>
▶ #include<stdlib.h>
▶ #define N 12
▶
▶ int add(int *a, int *b, int *c)
▶ {
▶   for(int i = 0; i < N; i++)
▶   {
▶     c[i] = a[i] + b[i];
▶   }
▶ }
▶ void charger(int *a)
▶ {
▶   int i;
▶   for (i = 0; i < N; ++i)
▶     a[i] = i;
▶ }
```

```
▶ int main()
▶ {
▶   int *a, *b, *c; // host (CPU) Copies of a, b, c
▶   int size = N * sizeof(int);
▶   /* Alloc space for host copies of a, b, c and setup
input values*/
▶   a = (int *)malloc(size);
▶   charger(a);
▶   b = (int *)malloc(size);
▶   charger(b);
▶   c = (int *)malloc(size);
▶
▶   add(a,b,c);
▶
▶   printf("Le vecteur C = A + B :\n");
▶   for (int i=0; i<N; i++)
▶     printf("%d |", c[i]);
▶   // Cleanup
▶   free(a);
▶   free(b);
▶   free(c);
▶   return 0;
▶ }
```

Programmation massive avec CUDA (3/5)

```
▶ // Prog. En CUDA
▶ #include<cuda.h>
▶ #include<device_launch_parameters.h>
▶ #include<cuda_device_runtime_api.h>
▶ #include<cuda_runtime_api.h>
▶ #include<stdio.h>
▶ #include<stdlib.h>
▶ #define N 12
▶
▶ __global__ void add(int *a, int *b, int *c)
▶ {
▶     c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
▶ }
▶
▶ void charger(int* a)
▶ {
▶     int i;
▶     for (i = 0; i < N; ++i)
▶         a[i] = i;
▶ }
▶
▶ int main(void)
▶ {
▶     int *a, *b, *c; // host (CPU) copies of a, b, c
▶     int *d_a, *d_b, *d_c; //device (GPU) copies of a,b,c
▶     int size = N * sizeof(int);
▶     // Alloc space for device copies of a, b, c
▶     cudaMalloc((void **)&d_a, size);
▶     cudaMalloc((void **)&d_b, size);
▶     cudaMalloc((void **)&d_c, size);
```

```
▶     // Alloc space for host copies of a, b, c and setup input values
▶     a = (int *)malloc(size);
▶     charger(a);
▶     b = (int *)malloc(size);
▶     charger(b);
▶     c = (int *)malloc(size);
▶
▶     // Copy inputs to device
▶     cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
▶     cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);
▶
▶     // Launch add() kernel on GPU with N blocks
▶     add<<<N,1>>>>(d_a, d_b, d_c);
▶
▶     // Copy result back to host
▶     cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);
▶     printf("Le vecteur C = A + B :\n");
▶     for (int i=0;i<N;i++)
▶         printf("%d |", c[i]);
▶
▶     // Cleanup
▶     free(a);
▶     free(b);
▶     free(c);
▶     cudaFree(d_a);
▶     cudaFree(d_b);
▶     cudaFree(d_c);
▶     return 0;
▶ }
```

Programmation massive avec CUDA (4/5)

- Un appel avec la syntaxe `<<< N,1>>>` permet de lancer N instances de la fonction noyau en parallèle
- Chaque instance de la fonction noyau est désignée par *un bloc*
- Chaque bloc contient un seul thread. La syntaxe `<<<N,1>>>` désigne donc N blocs de 1 thread chacun
- Au lieu de ça, on peut lancer 1 bloc de N threads, Ceci est possible en invoquant la fonction noyau avec la syntaxe `<<<1,N>>>`
- Chaque instance de la fonction noyau est désignée par *un thread*
- Chaque instance de la fonction noyau peut référencer l'indice de son thread en utilisant la syntaxe *threadIdx.x* (.x signifie qu'on utilise la représentation de threads sur une seule dimension)
- Les différentes valeurs de *threadIdx.x* peuvent être utilisées pour accéder aux différents éléments d'un tableau
- Nous pouvons alors utiliser les indices de blocs (*threadIdx.x*) pour réaliser l'addition (en parallèle) de deux vecteurs d'entiers

Programmation massive avec CUDA (5/5)

- ▶ Quelle est la manière la plus avantageuse entre les deux ?
 - ✓ *blockIdx.x (blocs parallèles) ou threadIdx.x (threads parallèles)*
 - ✓ *Autrement dit : la syntaxe <<<1,N>>> ou <<<N,1>>>*
 - ⇒ En l'absence de communications entre les threads, les deux manières sont équivalentes. Mais en présence de communications, on doit utiliser la deuxième manière
- ▶ Pourquoi ?
 - ⇒ Pour que les différents threads puissent communiquer et coopérer afin de réaliser une certaine tâche

Blocs + Threads parallèles en CUDA (1/3)

- Nous changeons le programme précédent pour utiliser les threads parallèles au lieu des blocs parallèles
- Pour ce faire, il suffit de changer la fonction noyau comme suit :

```
__global__ void add(int *a, int *b, int *c)  
{  
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];  
}
```

- Et de changer la manière dont elle est appelée comme suit :
add<<<1,N>>>(d_a, d_b, d_c);

Blocs + Threads parallèles en CUDA (2/3)

- Les deux manières d'exprimer le parallélisme (blocs parallèles et threads parallèles) ont un inconvénient commun qui réside dans le fait qu'elles sont limitées en terme du nombre maximum de threads que l'on peut implémenter
- Pour des contraintes de conception matérielles, les nombres maximum de blocs (peut ne pas dépasser 65535) et de threads par bloc (512, 1024, ...) sont limités
- Pour remédier à cela, on peut *combiner l'utilisation des blocs et threads parallèles*
- Le calcul de l'indice d'un élément d'un tableau est calculé en fonction des variables prédéfinies *blockIdx.x* et *threadIdx.x* comme suit : *int index = threadIdx.x + blockIdx.x * M* ; (M est le nombre de threads par bloc)

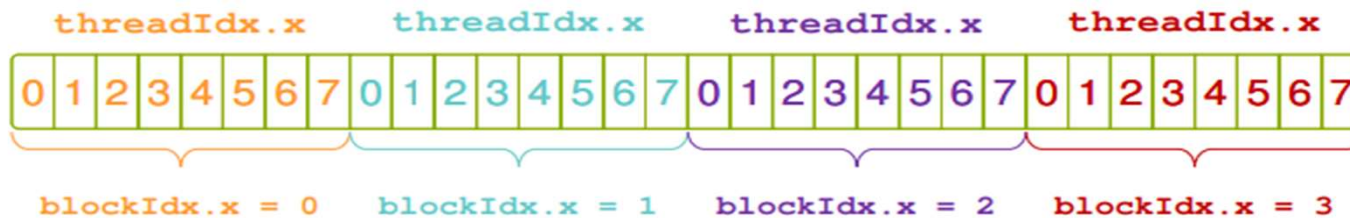
Blocs + Threads parallèles en CUDA (3/3)

➤ Blocs + threads parallèles

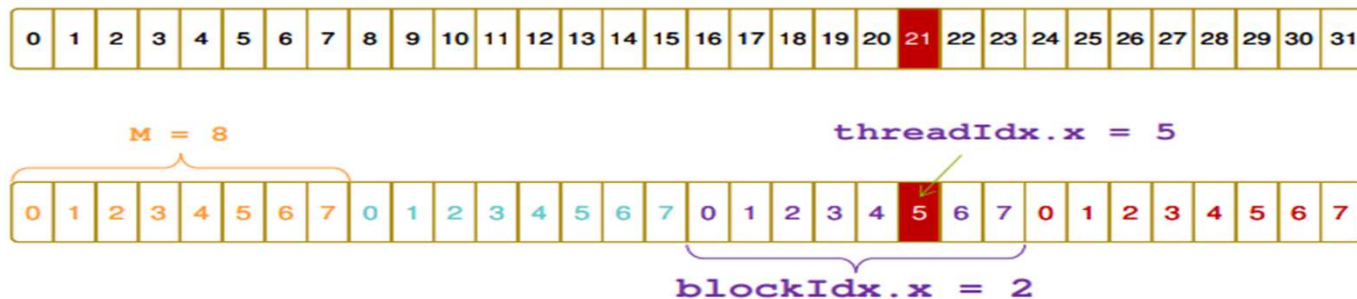
- ✓ Le calcul de l'indice d'un élément d'un tableau est calculé en fonction des variables prédéfinies *blockIdx.x* et *threadIdx.x* comme suit :

$int\ index = threadIdx.x + blockIdx.x * M$; (M est le nombre de threads par bloc)

- ✓ Exemple (M = 8)



- ✓ Quel thread va opérer sur l'élément en rouge (Avec M=8) ?



$$int\ index = threadIdx.x + blockIdx.x * M = 5 + 2 * 8 = 21$$

blockDim en CUDA (1/4)

- On peut utiliser la variable prédéfinie *blockDim.x* pour référencer au nombre (maximum) de threads contenus dans un bloc
- L'indice d'un élément du vecteur est ainsi donné par :
$$\text{int index} = \text{threadIdx.x} + \text{blockIdx.x} * \text{blockDim.x};$$
- La version combinée de la fonction kernel qui utilise les blocs et threads parallèles pour réaliser l'addition de deux vecteurs est donnée par :

```
__global__ void add(int *a, int *b, int *c)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    c[index] = a[index] + b[index];
}
```
- Dans un grand nombre d'applications, la taille des vecteurs n'est pas un multiple du nombre de threads par bloc *blockDim.x*

blockDim en CUDA (2/4)

- Afin d'éviter l'accès en dehors du vecteur (avec un indice supérieur ou égal à sa taille), nous devons effectuer un changement dans la fonction kernel ainsi que dans la manière dont elle est appelée comme suit :

```
__global__ void add(int *a, int *b, int *c)
{ int index = threadIdx.x + blockIdx.x * blockDim.x;
  if (index < N)
    c[index] = a[index] + b[index];
}
```

//Appel du kernel dans le main

add<<<(N + M-1)/M, M>>>(d_a, d_b, d_c)

- **Exemple 1** : pour $M=128$ et $N=127$, $N/M = 0$ (Lancer 0 bloc !), mais $(N+M-1)/M=1$
 - **Exemple 2** : pour $M=128$ et $N=40$, $\lll(N + M-1)/M, M\ggg = \lll 1, 128 \ggg$
- ⇒ on lance 128 threads, alors qu'on a seulement 40 éléments. Alors, on ne doit rien faire lorsque (indexe $\geq N$)

blockDim en CUDA (3/4)

- Dans le cas où la taille du vecteur dépasse le nombre maximal de threads autorisés, c-à-d, 65535 x 1024 (ce qui est déjà énorme), nous devons effectuer un autre changement dans la fonction noyau, comme suit :

```
__global__ void add(int *a, int *b, int *c)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    while (index < N)
    {
        c[index] = a[index] + b[index];
        index += blockDim.x * gridDim.x;
    }
}
```

- C-à-d, on incrémente à chaque fois l'indice du vecteur du nombre total de threads autorisés par le *GPU* (nombre de threads par bloc * nombre de blocs)

blockDim en CUDA (4/4)

```
▶ // Summing_vectors on threads_per_block
▶
▶ #include <stdio.h>
▶ #define N 100
▶
▶ __global__ void add(int *x, int *y, int *z)
▶ {
▶     int id = threadIdx.x + blockIdx.x * blockDim.x;
▶
▶     while (id < N)
▶     {
▶         z[id] = x[id] + y[id];
▶         id += blockDim.x * gridDim.x;
▶     }
▶ }
▶
▶ int main(void)
▶ {
▶     int x[N], *device_x;
▶     cudaMalloc((void **) &device_x, sizeof(int) * N);
▶
▶     int y[N], *device_y;
▶     cudaMalloc((void **) &device_y, sizeof(int) * N);
▶
▶     int z[N], *device_z;
▶     cudaMalloc((void **) &device_z, sizeof(int) * N);
▶
▶     for (int i = 0; i < N; i++) {
▶         x[i] = i;
▶         y[i] = -i;
▶     }
```

```
▶     cudaMemcpy(device_x, x, sizeof(int) * N, cudaMemcpyHostToDevice);
▶     cudaMemcpy(device_y, y, sizeof(int) * N, cudaMemcpyHostToDevice);
▶
▶     add<<<128, 128>>>(device_x, device_y, device_z);
▶
▶     cudaMemcpy(z, device_z, sizeof(int) * N, cudaMemcpyDeviceToHost);
▶
▶     int fail_count = 0;
▶     for (int i = 0; i < N; i++)
▶         if (z[i] != x[i] + y[i]) {
▶             printf("Error: %d + %d != %d\n", x[i], y[i], z[i]);
▶             fail_count += 1;
▶         }
▶     else
▶     {
▶         printf("%d + %d = %d\n", x[i], y[i], z[i]);
▶     }
▶
▶     if (fail_count)
▶         printf("Encountered %d errors\n", fail_count);
▶
▶     cudaFree(device_x);
▶     cudaFree(device_y);
▶     cudaFree(device_z);
▶
▶     return 0;
▶ }
```

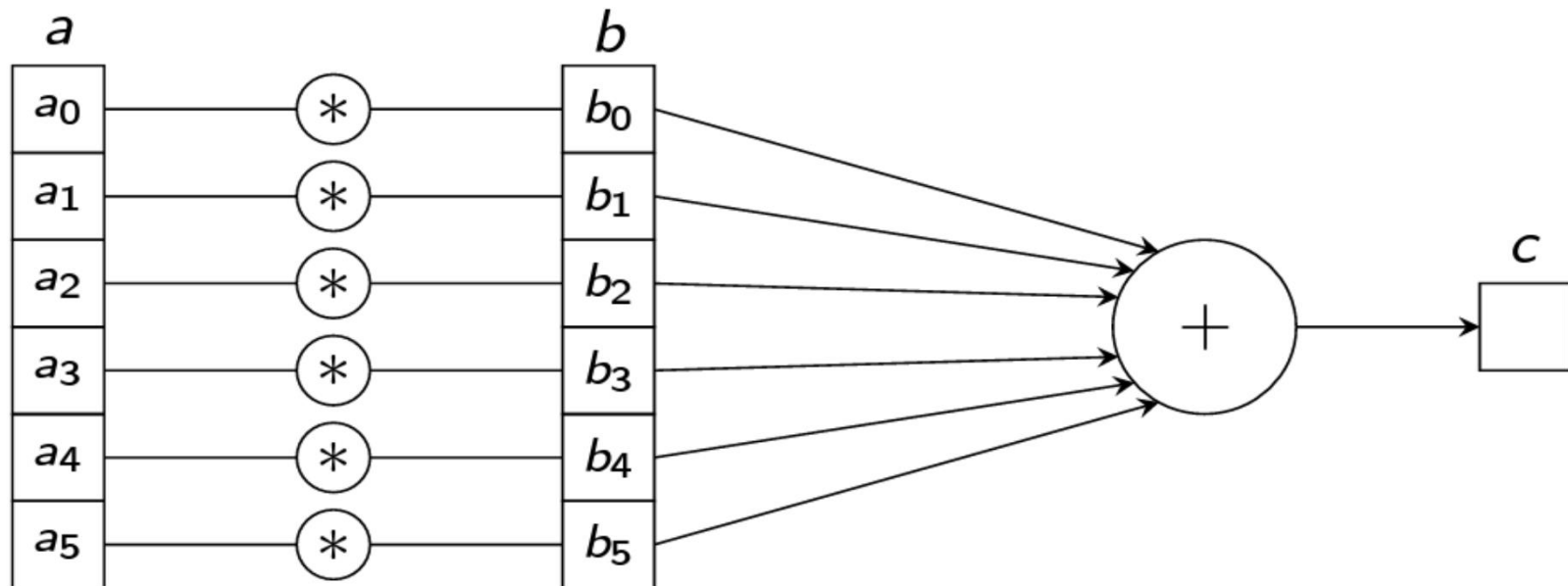
Coopération entre les threads en CUDA (1/1)

- Nous pouvons se contenter des blocs pour faire des calculs parallèles, l'utilisation des threads introduit un degré de complexité supplémentaire,
- Quel est l'intérêt d'utiliser les threads ?
- Contrairement aux blocs, les threads disposent de mécanismes qui leur permettent de :
 - ✓ Communiquer entre eux pour coopérer à la réalisation d'une certaine tâche
 - ✓ Synchroniser leurs actions pour éviter des comportements imprévisibles à cause de l'accès concurrent aux données «race condition»
- ⇒ Dans l'exemple vu précédemment (addition de deux vecteurs), on n'avait pas besoin de faire communiquer les threads entre eux (les traitements étaient totalement indépendants les uns des autres).
- Dans ce qui suit, nous allons étudier un exemple dans lequel les threads ont besoin de communiquer entre eux pour calculer une seule valeur qui est le *produit scalaire de deux vecteurs*.

Produit scalaire de deux vecteurs (1/9)

➤ 1 seul Block

➤ $c = (a_0, a_1, a_2, a_3, a_4, a_5) * (b_0, b_1, b_2, b_3, b_4, b_5)$
 $= a_0b_0 + a_1b_1 + a_2b_2 + a_3b_3 + a_4b_4 + a_5b_5$



Produit scalaire de deux vecteurs (2/9)

➤ 1 seul Block

```
__global__ void dot (int *a, int *b, int *c)
{ // chaque thread calcule le produit d'une paire
  int temp = a[ threadIdx .x] * b[ threadIdx .x ];
}
```

➤ Le calcul est local au processus

- ✓ Les variables temp ne sont pas accessibles aux autres processus
- ✓ Mais il faut partager les données pour faire la somme finale

➤ Partage des données entre les threads d'un même bloc :

- ✓ Les threads d'un block partagent une zone mémoire appelée shared memory déclarée avec *__shared__*

➤ Des blocks sur le même **SM** (Streaming Multiprocessors) partagent la même mémoire partagée **globale**.

➤ Pour une mémoire de 48KB avec N blocks sur le même SM, chaque block possédera 48/N de mémoire partagée

- 1 thread = 1 cœur
- 1 block = 1 SM
- 1 block est exécuté sur 1 SM
- Les blocks sont exécutés dans n'importe quel ordre, séquentiellement ou en parallèle
- Contrairement aux blocks, les threads peuvent communiquer ou se synchroniser à l'intérieur d'un block

Produit scalaire de deux vecteurs (3/9)

➤ 1 seul Block : shared memory

```
#define N 512 //taille du tableau
__global__ void dot (int *a, int *b, int *c)
{
    __shared__ int tmp[N]
    // chaque thread calcule le produit d'une paire
    int temp[ threadIdx .x] = a[ threadIdx .x] * b[ t hreadIdx . x ];
    //Le thread 0 effectue la somme
    if (0 == threadIdx .x)
    {
        int sum = 0;
        for ( int i = 0; i < N; i++)
            sum = sum + temp[ i ];
        *c = sum;
    }
}
```

Produit scalaire de deux vecteurs (4/9)

➤ 1 seul Block : Synchronisation des threads d'un même block

✓ Synchronise uniquement les threads d'un même block : fonction `__syncthreads()`

✓ `__syncthreads()` : C'est une barrière pour les threads d'un seul block

#define N 512 //taille du tableau

*__global__ void dot (int *a, int *b, int *c)*

{ __shared__ int temp[N]

// chaque thread calcule le produit d'une paire

*temp[threadIdx .x] = a[threadIdx .x] * b[threadIdx . x] ;*

// Synchronisation pour être sur que tous les threads ont fini

__syncthreads () ;

//Le thread 0 effectue la somme

if (0 == threadIdx .x)

{ int sum = 0;

for (int i = 0; i < N; i++)

*sum = sum + temp[i]; *c = sum;*

}

}

Produit scalaire de deux vecteurs (5/9)

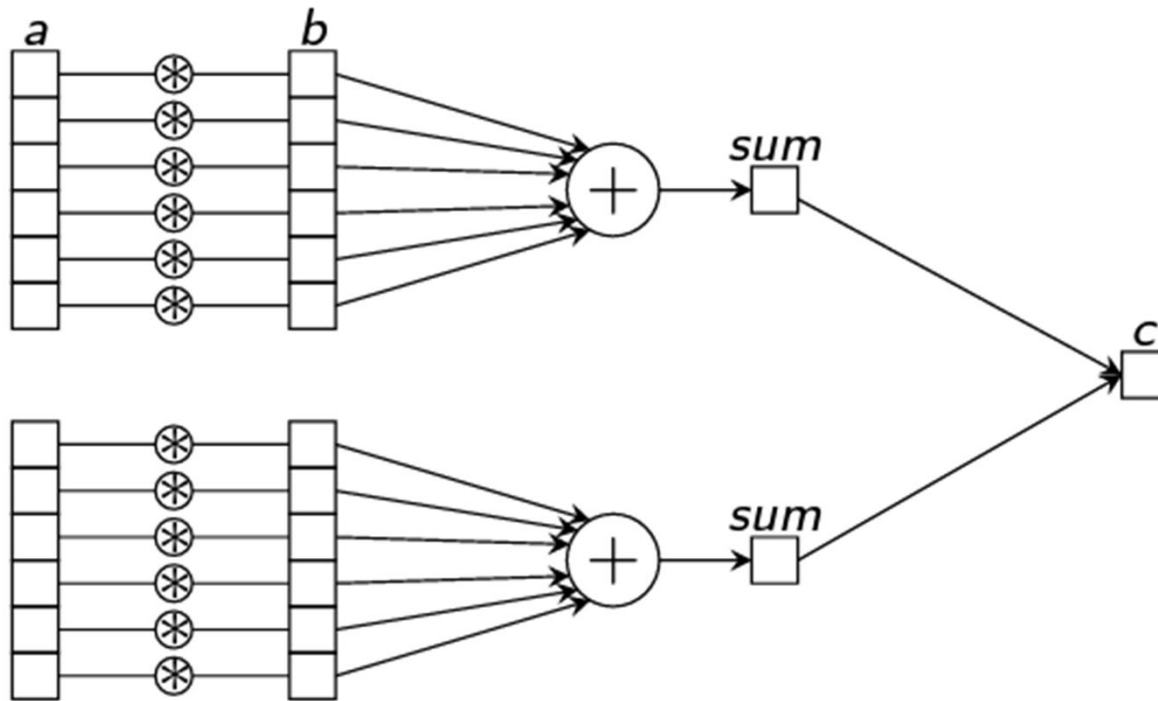
➤ //Dot Vect en Cuda

```
➤ #include<cuda.h>
➤ #include<device_launch_parameters.h>
➤ #include<cuda_device_runtime_api.h>
➤ #include<cuda_runtime_api.h>
➤ #include<stdio.h>
➤ #include<stdlib.h>
➤
➤ #define N 12 //taille du tableau
➤
➤ __global__ void dot( int*a, int*b, int*c )
➤ {
➤     // Shared memory for results of multiplication
➤     __shared__ int temp[N];
➤     temp[threadIdx.x] = a[threadIdx.x] * b[threadIdx.x];
➤
➤     // __syncthreads();
➤
➤     // Thread 0 sums the pairwise products
➤     if( threadIdx.x == 0 )
➤     {
➤         int sum = 0;
➤         for( int i= 0; i< N; i++ )
➤             sum += temp[i];
➤         *c = sum;
➤     }
➤ }
➤
➤ void charger(int* a)
➤ {
➤     int i;
➤     for (i = 0; i < N; ++i)
➤         a[i] = i;
➤ }
```

```
➤ int main (void)
➤ {
➤     int *a, *b, *c; // copies of a, b, c
➤     int *dev_a, *dev_b, *dev_c; // device copies of a, b, c
➤     int size = N * sizeof( int ); // we need space for N integers
➤     // allocate device copies of a, b, c
➤     cudaMalloc( (void**)&dev_a, size );
➤     cudaMalloc( (void**)&dev_b, size );
➤     cudaMalloc( (void**)&dev_c, sizeof( int ) );
➤     a = (int *)malloc( size );
➤     b = (int *)malloc( size );
➤     c = (int *)malloc( sizeof( int ) );
➤     charger( a );
➤     charger( b );
➤     // copy inputs to device
➤     cudaMemcpy( dev_a, a, size, cudaMemcpyHostToDevice );
➤     cudaMemcpy( dev_b, b, size, cudaMemcpyHostToDevice );
➤     // Launch dot() kernel with 1 block and N threads
➤     dot<<< 1, N >>>( dev_a, dev_b, dev_c );
➤     // copy device result back to host copy of c
➤     cudaMemcpy( c, dev_c, sizeof( int ), cudaMemcpyDeviceToHost );
➤
➤     printf("Vector's dot = %d", *c);
➤
➤     free( a ); free( b ); free( c );
➤     cudaFree( dev_a );
➤     cudaFree( dev_b );
➤     cudaFree( dev_c );
➤     return 0;
➤ }
```

Produit scalaire de deux vecteurs (6/9)

➤ Plusieurs Blocks



Produit scalaire de deux vecteurs (7/9)

➤ Plusieurs Blocks

- ✓ Le calcul se fait dans la mémoire globale
- ✓ plusieurs *thread 0* peuvent y accéder en même temps
- ✓ Les opérations atomiques :
 - Les opération de lecture, modification et écriture sont ininterrompibles
 - Plusieurs opérations atomiques possibles avec CUDA :
 - *atomicAdd()* : Addition atomique
 - *atomicSub()* : Soustraction atomique
 - *atomicMin()* : Minimum atomique
 - *atomicMax()* : Maximum atomique
 - *atomicInc()* : Incrémentation atomique
 - *atomicDec()* : Décrémentement atomique
 - *atomicExch()* : Echange atomique
 - *atomicCAS()* : (CAS : Compare And Swap)

- *atomicCAS(A,B,C)* : Compare B avec A dans la mémoire globale ou partagée (shared). En cas d'égalité, elle met dans A la valeur de C.
- Les opérations de lecture-Ecriture-Modification sont garanties sans interférences avec d'autres threads

Produit scalaire de deux vecteurs (8/9)

➤ Plusieurs Block

```
#defin N (2048 * 2048)
#define TH_PER_BLOCK 512 // taille du tableau
__global__ void dot (int *a, int *b, int *c)
{
    __shared__ int tmp[TH_PER_BLOCK]
    // chaque thread calcule le produit d'une paire
    int indice = threadIdx .x + blockIdx .x * blockDim .x ;
    temp[ indice ] = a[ indice ] * b[ indice ];
    // Synchronisation (dans le block)
    __syncthreads () ;
    //Le thread 0 effectue la somme
    if (0 == threadIdx .x)
    { int sum = 0;
      for ( int i = 0; i < TH_PER_BLOCK; i++)
          sum = sum + temp[ i ];
      atomicAdd(*c , sum) ;
    }
}
```

Produit scalaire de deux vecteurs (9/9)

> Le main (plusieurs blocks)

```
> #include<cuda.h>
> #include<device_launch_parameters.h>
> #include<cuda_device_runtime_api.h>
> #include<cuda_runtime_api.h>
> #include<stdio.h>
> #include<stdlib.h>
>
> #define N 12
> #define THREADS_PER_BLOCK 4
>
> __global__ void dot( int*a, int*b, int*c ) {
>     __shared__ int temp[THREADS_PER_BLOCK];
>
>     int index = threadIdx.x + blockIdx.x * blockDim.x;
>     temp[threadIdx.x] = a[index] * b[index];
>     __syncthreads();
>
>     if( threadIdx.x == 0 ) {
>         int sum = 0;
>         for( int i= 0; i< THREADS_PER_BLOCK; i++ )sum += temp[i];
>         atomicAdd(c,sum);
>     }
> }
>
> void charger(int* a)
> {
>     int i;
>     for( i= 0; i < N; i++)
>         a[i] = i;
> }
```

```
> int main( void )
> {
>     int *a, *b, *c; // copies of a, b, c
>     int *dev_a, *dev_b, *dev_c; // device copies of a, b, c
>     int size = N * sizeof( int ); // we need space for N integers
>     // allocate device copies of a, b, c
>     cudaMalloc( (void**)&dev_a, size );
>     cudaMalloc( (void**)&dev_b, size );
>     cudaMalloc( (void**)&dev_c, size );
>     a = (int *)malloc( size );
>     b = (int *)malloc( size );
>     c = (int *)malloc( size );
>     charger( a );
>     charger( b );
>     // copy inputs to device
>     cudaMemcpy( dev_a, a, size, cudaMemcpyHostToDevice );
>     cudaMemcpy( dev_b, b, size, cudaMemcpyHostToDevice );
>
>     // launch dot() kernel with 1 block and N threads
>     dot<<<ceil(N/THREADS_PER_BLOCK), THREADS_PER_BLOCK>>>( dev_a, dev_b,
dev_c);
>
>     // copy device result back to host copy of c
>     cudaMemcpy( c, dev_c, size, cudaMemcpyDeviceToHost );
>
>     printf("Vector's dot = %d ", *c);
>
>     free( a ); free( b ); free( c );
>     cudaFree( dev_a );
>     cudaFree( dev_b );
>     cudaFree( dev_c );
>     return 0;
> }
```

Temps d'exécution du kernel (1/2)

➤ Utiliser CPU Timer

- ✓ Les transferts de données sont synchrones
- ✓ L'appel au kernel n'est pas synchrone (Asynchrone)

CPU Timer :

```
#include <time,h>
cudaMemcpy (gpu_a,a, size ,cudaMemcpyHostToDevice) ;
cudaMemcpy (gpu_b,b, size ,cudaMemcpyHostToDevice) ;
t 1 = myCPUTimer ( ) ;
add <<< N/TH_PER_BLOCK,THREAD_PER_BLOCK >>> ( gpu_a, gpu_b, gpu_c) ;
c u d a D e v i c e S y n c h r o n i z e ( ) ;
t 2 = myCPUTimer ( ) ;
cudaMemcpy(c , gpu_c, size , cudaMemcpyDeviceToHost) ;
Printf("Exec. Time = %d\n", T2-T1);
```

Temps d'exécution du kernel (2/2)

➤ Utiliser GPU Timer

- ✓ Les opérations sont séquentielles sur le *GPU*
- ✓ On utilise CUDA event API (*cudaEvent_t*)
- On va mesurer la bande passante effective de la mémoire sur la fonction qui calcule : $z = ax + y$ où x , y et z sont trois tableaux et a est un scalaire
- ✓ Enregistrement des événements : *cudaEventRecord(start, 0);* et *cudaEventRecord(stop, 0);*
- ✓ Synchronisation *CPU/GPU* : *cudaEventSynchronize(stop);*
- ✓ Calcul du temps écoulé entre les deux instants : *cudaEventElapsedTime(&elapsedTime, start, stop);*
- ✓ Destruction (libération) des événements : *cudaEventDestroy(&start);* et *cudaEventDestroy(&stop);*

```
#include<stdio.h>
#include<cuda.h>
__global__ void saxpy(int n, float a, float *x, float *y)
{ int i = blockIdx.x*blockDim.x + threadIdx.x;
  if (i < n) y[i] = a*x[i] + y[i]; //(z = ax + y)
}
int main(void)
{ int N = 20 * (1 << 20);
  float *x, *y, *d_x, *d_y;
  x = (float*)malloc(N*sizeof(float));
  y = (float*)malloc(N*sizeof(float));
  cudaMalloc(&d_x, N*sizeof(float));
  cudaMalloc(&d_y, N*sizeof(float));
  for (int i = 0; i < N; i++)
  { x[i] = 1.0f;
    y[i] = 2.0f;
  }
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
```

```
  cudaMemcpy(d_x, x, N*sizeof(float), cudaMemcpyHostToDevice);
  cudaMemcpy(d_y, y, N*sizeof(float), cudaMemcpyHostToDevice);
  cudaEventRecord(start);
  // Perform Bande on 1M elements
  saxpy<<<(N+511)/512, 512>>>(N, 2.0f, d_x, d_y);
  cudaEventRecord(stop);
  cudaMemcpy(y, d_y, N*sizeof(float), cudaMemcpyDeviceToHost);
  cudaEventSynchronize(stop);
  float milliseconds = 0;
  cudaEventElapsedTime(&elapsedTime, start, stop);
  float maxError = 0.0f;
  for (int i = 0; i < N; i++)
  { maxError = max(maxError, abs(y[i]-4.0f)); }
  printf("Max error: %fn", maxError);
  printf("Elapsed time %3.1 f ms\n", elapsedTime);
}
```

Allez plus loin en CUDA

➤ Les optimisations de base

- ✓ Throughput de la mémoire :
 - ❑ Minimiser les transfert *Host*↔*Device*
 - ❑ Minimiser les transferts mémoire *globale*↔*Device*
 - ⇒ Favoriser la mémoire partagée et les caches
- ✓ La mémoire globale
 - ❑ **Coalescence** : grouper (coalescer) les accès contigus en un seul accès à différentes location de la DRAM
 - ⇒ Éviter les accès non alignés
 - ❑ L'accès aux **banks** est simultané
 - ⇒ La mémoire est partagée en modules de tailles égales appelés **bank**
 - ⇒ Toute lecture/écriture de n adresses dans n banks différents est simultané
- ✓ Conflits de banks (**Bank conflict**)
 - ❑ Si deux accès sont dans différentes adresses du même bank, il y a conflit
 - ❑ La mémoire partagée est rapide tant qu'il n'y a pas de bank conflict
- ✓ Application : Produit matriciel

➤ Les streams

- ✓ Un **stream** : Une séquence d'opérations (comme une file d'attente pour le device)
- ✓ Les opérations sur un même stream sont ordonnées (FIFO) et synchrones
- ✓ Les opérations dans des streams différents peuvent s'intercaler et s'exécuter en parallèle

➤ MPI + CUDA

➤ Mémoire unifiée CPU + GPU

Merci pour
votre
attention

